

Algorithms and Data Structures I.

Practice 2

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

February 27, 2024

Content in this Practice I

1 Review last week

- Max Selection Sort - example
- Bubble Sort - example
- Improved Bubble Sort - example
- Insertion Sort - Example
- Insertion Sort - Analysis
- Stable sorting algorithms

2 Divide and Conquer algorithms

- Divide and Conquer algorithms
- Merge Sort
- Merge Sort - Divide
- Merge Sort - Representation example

Content in this Practice II

- Merge Sort - Merge example
- Merge Sort - Merge structogram
- Merge Sort - Summary

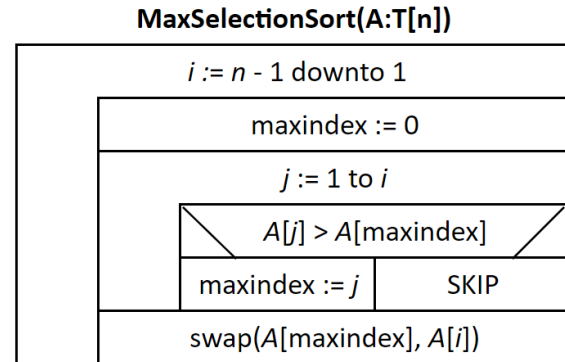
Review last week
Divide and Conquer algorithms

Max Selection Sort - example
Bubble Sort - example
Improved Bubble Sort - example
Insertion Sort - Example
Insertion Sort - Analysis
Stable sorting algorithms

Review last week

Max Selection Sort - example

Array:	$\langle 5, 4, 2, 8, 3 \rangle$
1. iteration:	$\langle \underline{5}, 4, 2, \underline{8}, \underline{3} \rangle$
2. iteration:	$\langle \underline{5}, 4, 2, \underline{3} 8 \rangle$
3. iteration:	$\langle 3, \underline{4}, \underline{2} 5, 8 \rangle$
4. iteration:	$\langle \underline{3}, \underline{2} 4, 5, 8 \rangle$
5. iteration:	$\langle \underline{2} 3, 4, 5, 8 \rangle$
Sorted array:	$\langle 2, 3, 4, 5, 8 \rangle$

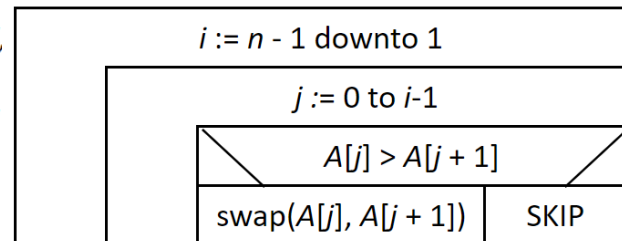


Bubble Sort - example

0	1	2	3	4	swap
3	5	2	4	1	0
3	5	2	4	1	1
3	2	5	4	1	1
3	2	4	5	1	1
3	2	4	1	5	1. iteration end, 5 is sorted
3	2	4	1	5	1
2	3	4	1	5	0
2	3	4	1	5	1
2	3	1	4	5	2. iteration end, 4 and 5
2	3	1	4	5	0
2	3	1	4	5	1
2	1	3	4	5	3. iteration end,
2	1	3	4	5	1
1	2	3	4	5	4. iteration end

Overall swap: 7
Overall comparisons: 10

BubbleSort(A :T[n])

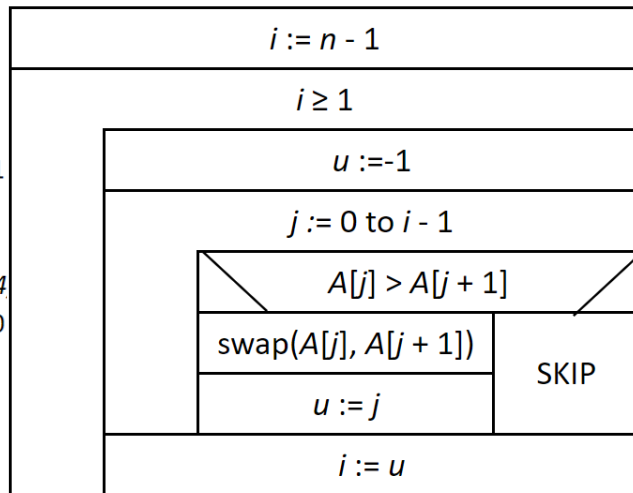


Improved Bubble Sort - example

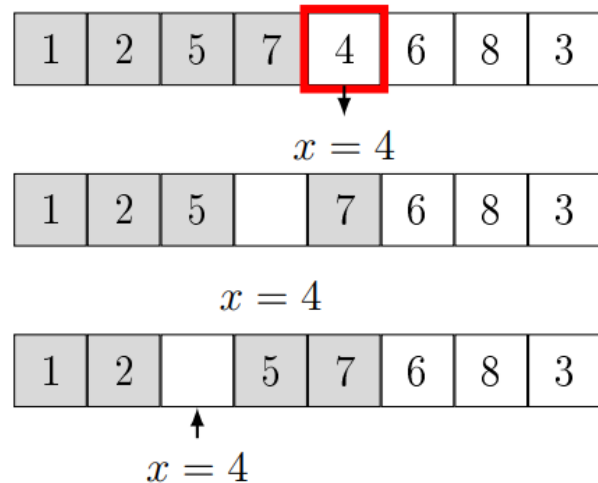
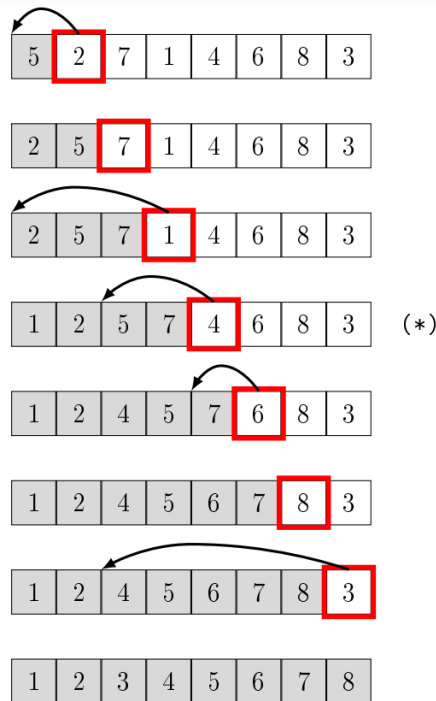
0	1	2	3	4	swap	
2	3	1	4	5	0	
2	3	1	4	5	1	u=1
2	1	3	4	5	0	
2	1	3	4	5	0	
2	1	3	4	5	1	1. iteration end 3,4
2	1	3	4	5	1	u=0
1	2	3	4	5		terminate

Overall swap: 2
Overall comparisons: 5

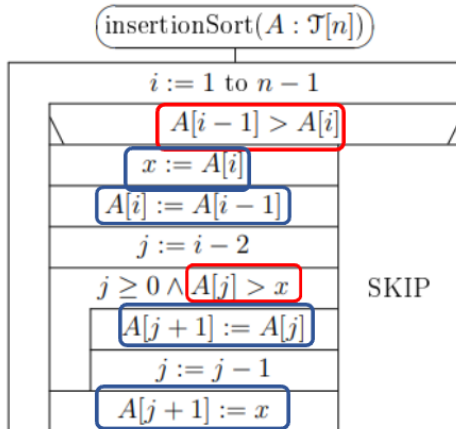
ImprovedBubbleSort(A:T[n])



Insertion Sort - Example



Insertion Sort - Analysis



0

1

2

3

4

5

6

7

comp move

A

4

3

1

3'

5

2

6

4'

0

1

2

3

5

6

7

8

>

i

1. iteration

A

4

3

1

3'

5

2

6

4'

1

3

>

>

i

2. iteration

A

3

4

1

3'

5

2

6

4'

2

4

£

>

i

3. iteration

A

1

3

4

3'

5

2

6

4'

2

3

£

i

4. iteration

A

1

3

3'

4

5

2

6

4'

1

0

£

>

>

>

>

i

5. iteration

A

1

3

3'

4

5

2

6

4'

5

6

£

i

6. iteration

A

1

2

3

3'

4

5

6

4'

1

0

£

>

>

7. iteration

A

1

2

3

3'

4

5

6

4'

3

4

Overall:

A

1

2

3

3'

4

4'

5

6

15

20

Stable sorting algorithms

- A sorting algorithm is stable if given multiple records with the same values it keeps their original ordering.
- $\langle 1, 3, 1', 2, 6 \rangle \rightarrow \langle 1, 1', 2, 3, 6 \rangle$
- Insertion sort $\rightarrow$ **Stable**
- Bubble sort $\rightarrow$ **Stable**
- Maximum selection sort $\rightarrow$ **Not Stable**
Simple case example: $\langle 1, 2, 6, 5, 6' \rangle \rightarrow$ *watch out for the comparison operator for each direction of the inner loop*
 $\rightarrow$ Can be stable
Difficult case example: $\langle 1, 7, 6, 5, 6' \rangle \rightarrow$ *instead of swap, push the data* $\rightarrow$ Can be stable

Divide and Conquer algorithms

Divide and Conquer algorithms

- *A divide-and-conquer algorithm breaks down a large problem into smaller subproblems, which themselves may be broken down into even smaller subproblems, and so forth. The recursion bottoms out when it reaches a base case and the subproblem is small enough to solve directly without further recursing.*
- A divide-and-conquer algorithm:
 - **Divide** the problem into one or more subproblems that are smaller instances of the same problem.
 - **Conquer** the subproblems by solving them recursively.
 - **Combine** the subproblem solutions to form a solution to the original problem.

Merge Sort

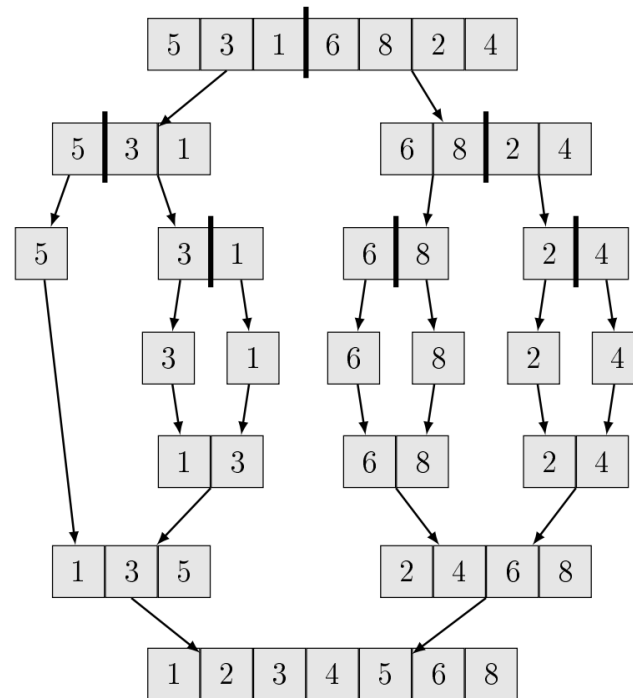
- Main idea:

Let's imagine we have two already sorted arrays. Is there a good way of creating a new combined sorted array from it?

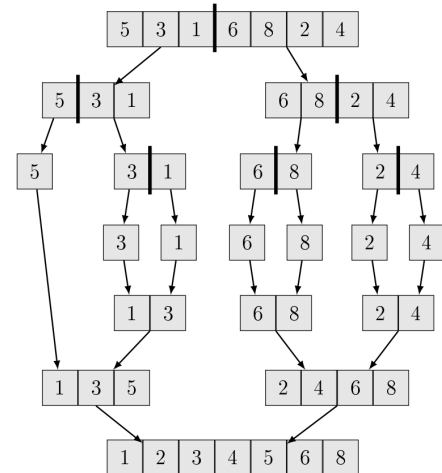
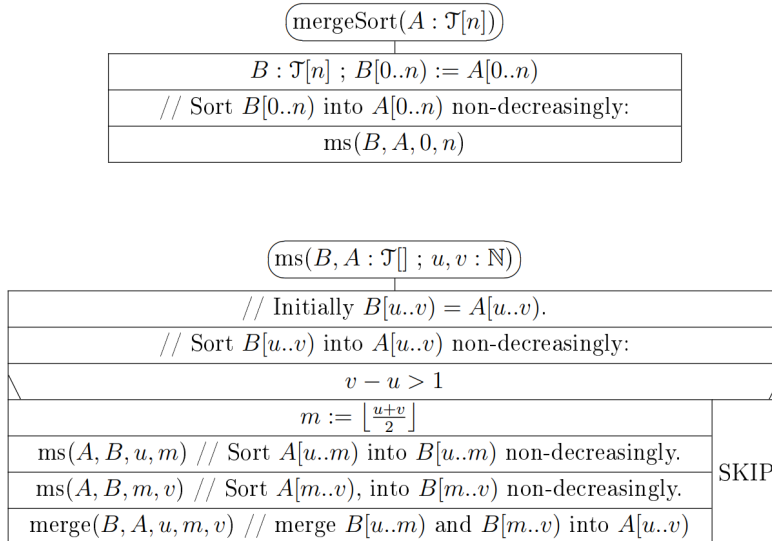
- Merge Sort:

Merge sort is a recursive algorithm that continuously splits the array in half until it cannot be further divided i.e., the array has only one element left (an array with one element is always sorted). Then the sorted subarrays are merged into one sorted array.

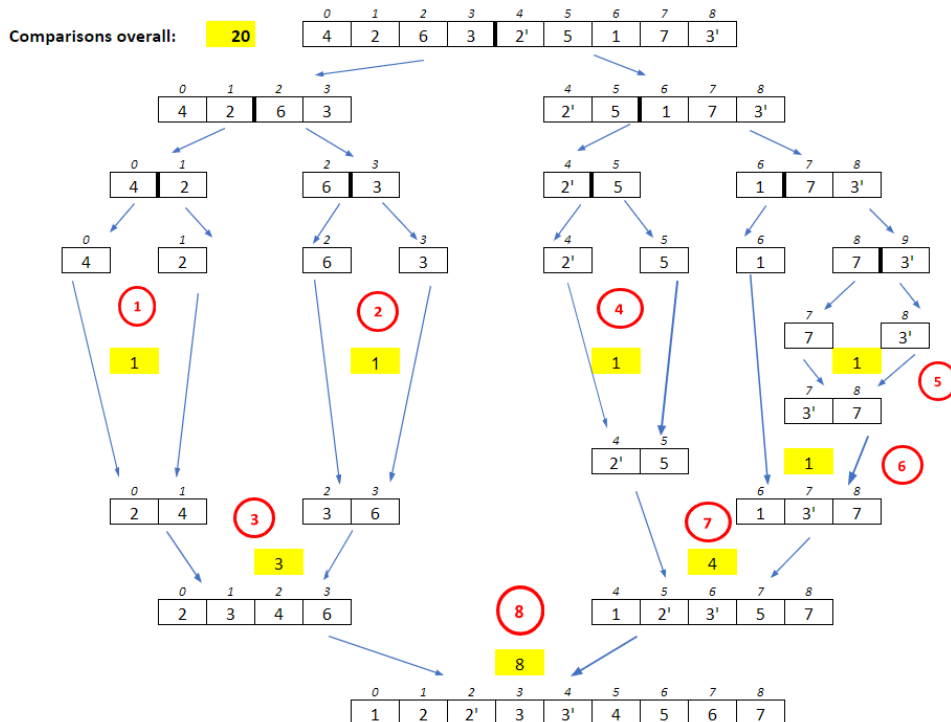
Merge Sort - Representation



Merge Sort - Divide



Merge Sort - Representation example



Merge Sort - Representation example

1

START		0	1	2	3	4	5	6	7	8
A		4	2	6	3	2'	5	1	7	3'
B		4	2	6	3	2'	5	1	7	3'

2

START		0	1	2	3	4	5	6	7	8
A		2	4	6	3	2'	5	1	7	3'
B		4	2	6	3	2'	5	1	7	3'

3

START		0	1	2	3	4	5	6	7	8
A		2	4	3	6	2'	5	1	7	3'
B		4	2	6	3	2'	5	1	7	3'

4

START		0	1	2	3	4	5	6	7	8
A		2	4	3	6	2'	5	1	7	3'
B		2	3	4	6	2'	5	1	7	3'

merge(B,A,0,1,2) END comparisons

END		0	1	2	3	4	5	6	7	8
A		2	4	6	3	2'	5	1	7	3'
B		4	2	6	3	2'	5	1	7	3'

1

(from B to A)

merge(B,A,2,3,4) END

END		0	1	2	3	4	5	6	7	8
A		2	4	3	6	2'	5	1	7	3'
B		4	2	6	3	2'	5	1	7	3'

1

(from B to A)

merge(A,B,0,2,4) END

END		0	1	2	3	4	5	6	7	8
A		2	4	3	6	2'	5	1	7	3'
B		2	3	4	6	2'	5	1	7	3'

3

(from A to B)

merge(B,A,4,5,6) END

END		0	1	2	3	4	5	6	7	8
A		2	4	3	6	2'	5	1	7	3'
B		2	3	4	6	2'	5	1	7	3'

1

(from B to A)

Merge Sort - Representation example

5

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	2	3	4	6	2'	5	1	7	3'

START

6

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	2	3	4	6	2'	5	1	3'	7

START

7

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	3'	7
B	2	3	4	6	2'	5	1	3'	7

START

8

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	3'	7
B	2	3	4	6	1	2'	3'	5	7

merge(A,B,7,8,9)

(from A to B)

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	2	3	4	6	2'	5	1	3'	7

1

END

merge(B,A,6,7,9)

(from B to A)

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	3'	7
B	2	3	4	6	2'	5	1	3'	7

1

END

merge(A,B,4,6,9)

(from A to B)

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	3'	7
B	2	3	4	6	1	2'	3'	5	7

4

END

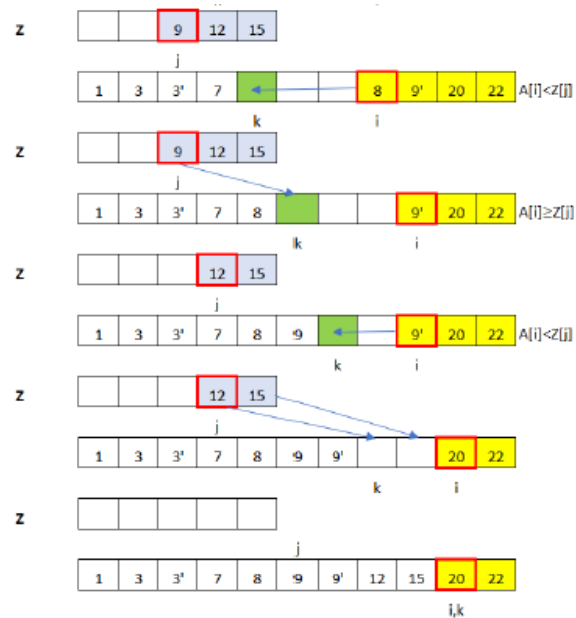
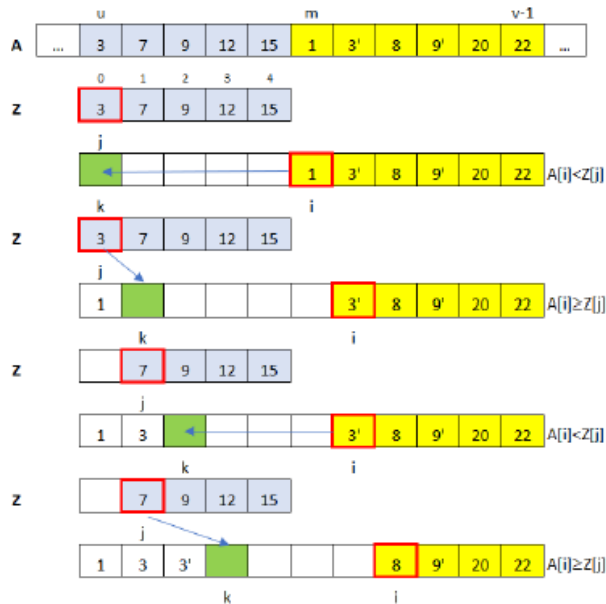
merge(B,A,0,4,9)

(from B to A)

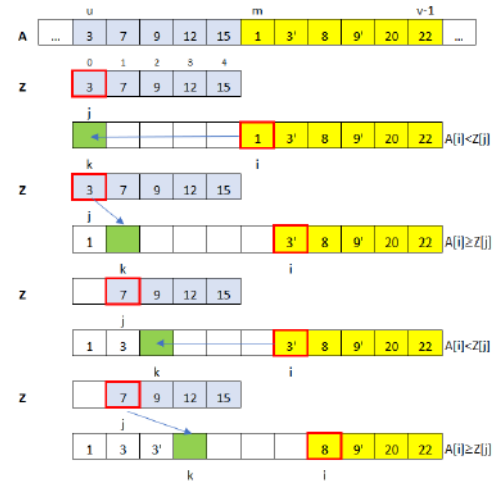
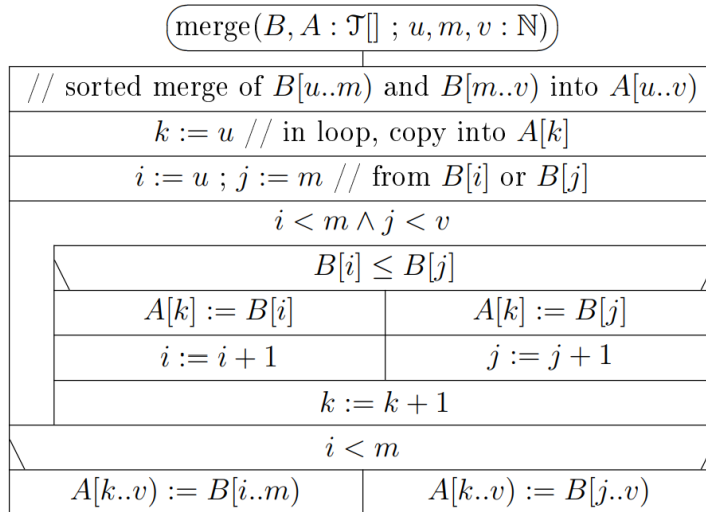
	0	1	2	3	4	5	6	7	8
A	1	2	2'	3	3'	4	5	6	7
B	2	3	4	6	1	2'	3'	5	7

8

Merge Sort - Merge example



Merge Sort - Merge structogram



Merge Sort - Summary

Time complexity:

- $mT(n), AT(n), MT(n) \in O(n * \log(n))$

Space complexity

- $O(n)$
- We need to build a temporary array to merge the sorted arrays.

Stable: The sorting algorithms maintain the relative order of the duplicate data.

Review last week
Divide and Conquer algorithms

Divide and Conquer algorithms
Merge Sort
Merge Sort - Divide
Merge Sort - Representation example
Merge Sort - Merge example
Merge Sort - Merge structogram
Merge Sort - Summary

Ending

Thank you for your attention!